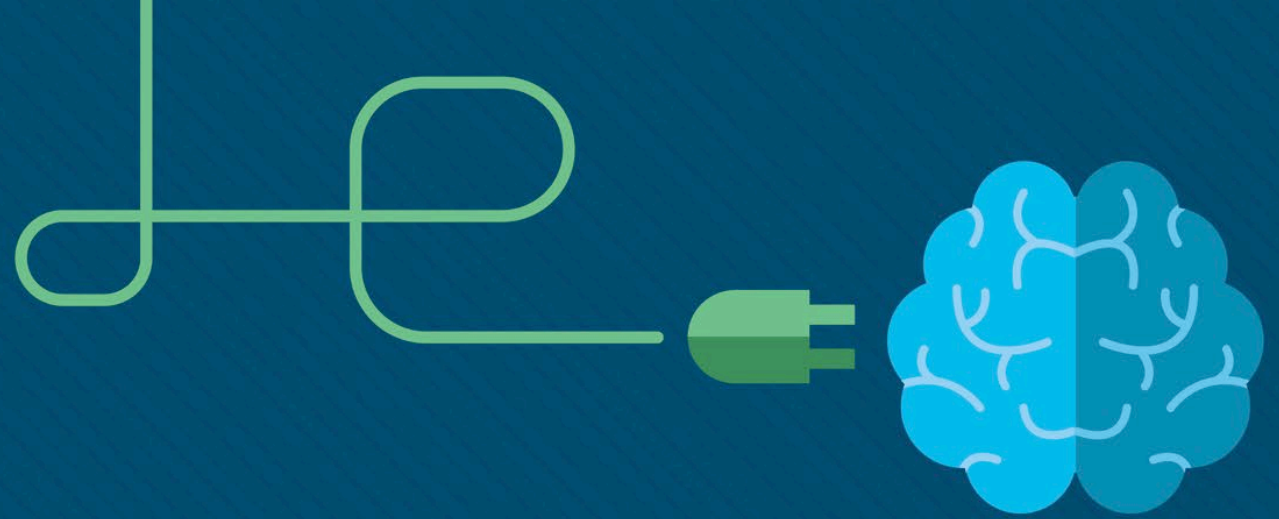




Week 2

RH124 Chapter 3: Managing files from the Command Line

RH124 Chapter 4: Getting Help



1 The File-system Hierarchy

- The Linux system stores all files on file systems, which are organized into a single inverted tree known as a file-system hierarchy.
- Logically, the filesystem is perceived by users to be a hierarchical order of directories and files with the root “/” directory at the top of the directory tree.

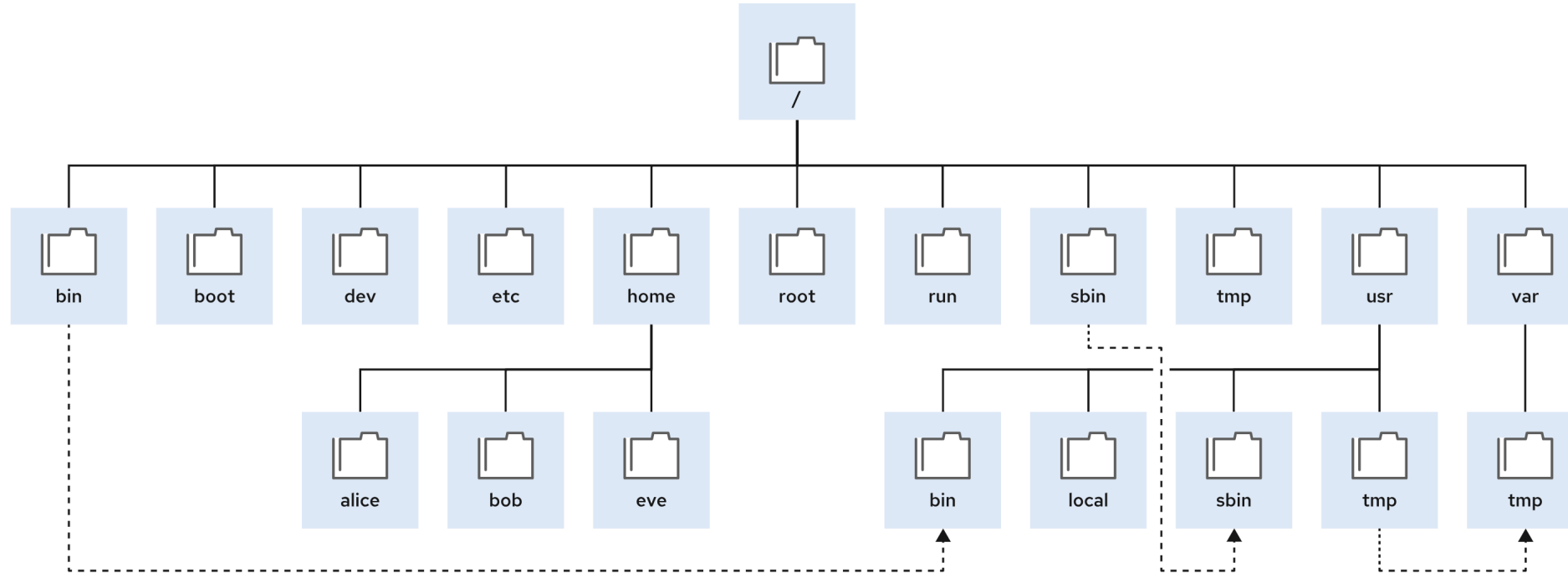


Figure 3.1: Significant file-system directories in Red Hat Enterprise Linux 9

The following table lists some of the significant directories on the system by name and purpose.

Location	Purpose
/boot	Files to start the boot process.
/dev	Special device files that the system uses to access hardware.
/etc	System-specific configuration files.
/home	Home directory, where regular users store their data and configuration files.
/root	Home directory for the administrative superuser, root.
/run	Runtime data for processes that started since the last boot. This data includes process ID files and lock files. The contents of this directory are re-created on reboot. This directory consolidates the /var/run and /var/lock directories from earlier versions of Red Hat Enterprise Linux.
/tmp	A world-writable space for temporary files. Files that are not accessed, changed, or modified for 10 days are deleted from this directory automatically. The /var/tmp directory is also a temporary directory, in which files that are not accessed, changed, or modified in more than 30 days are deleted automatically.
/usr	Installed software, shared libraries, including files, and read-only program data. Significant subdirectories in the /usr directory include the following commands: • /usr/bin: User commands • /usr/sbin: System administration commands • /usr/local: Locally customized software
/var	System-specific variable data should persist between boots. Files that dynamically change, such as databases, cache directories, log files, printer-spoiled documents, and website content, might be found under /var.

2 Absolute and Relative Paths

- Absolute path e.g. `/usr/sbin` – unique location in the file system hierarchy. Always starts with a `/`.

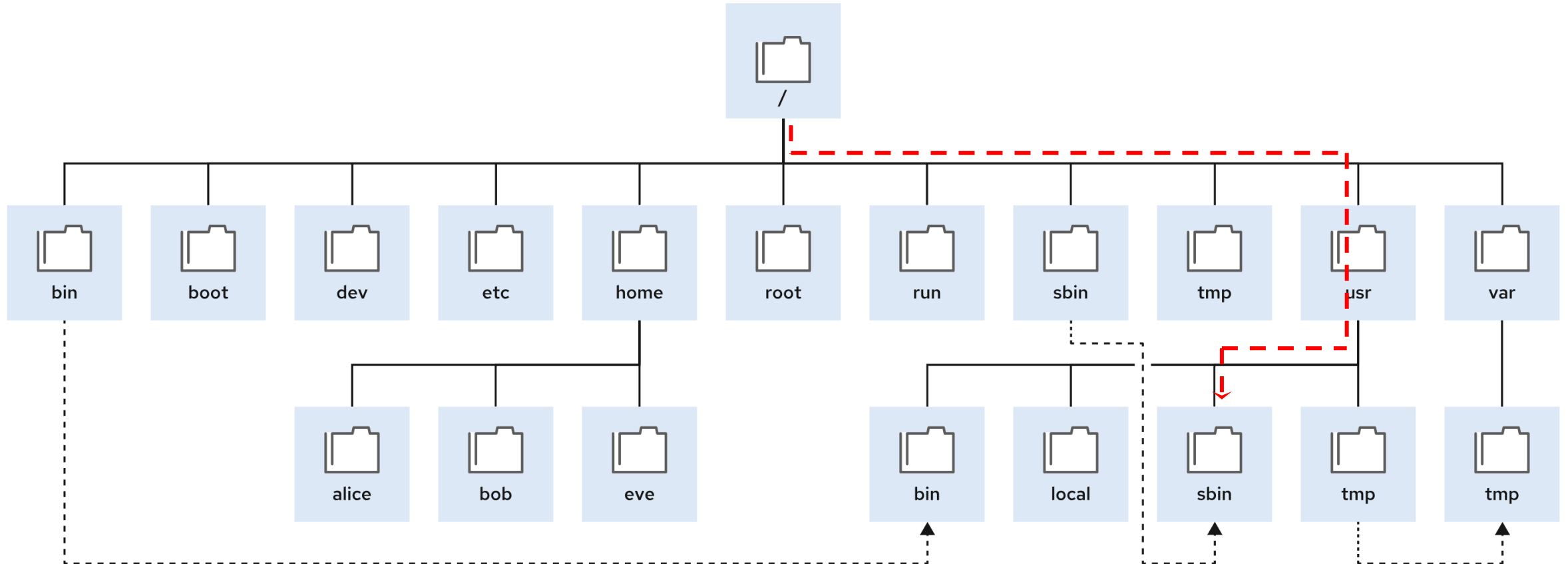


Figure 3.1: Significant file-system directories in Red Hat Enterprise Linux 9

2 Absolute and Relative Paths (cont'd)

- Relative path e.g. eve – you will only know which directory is referenced when you know the current work directory of the user

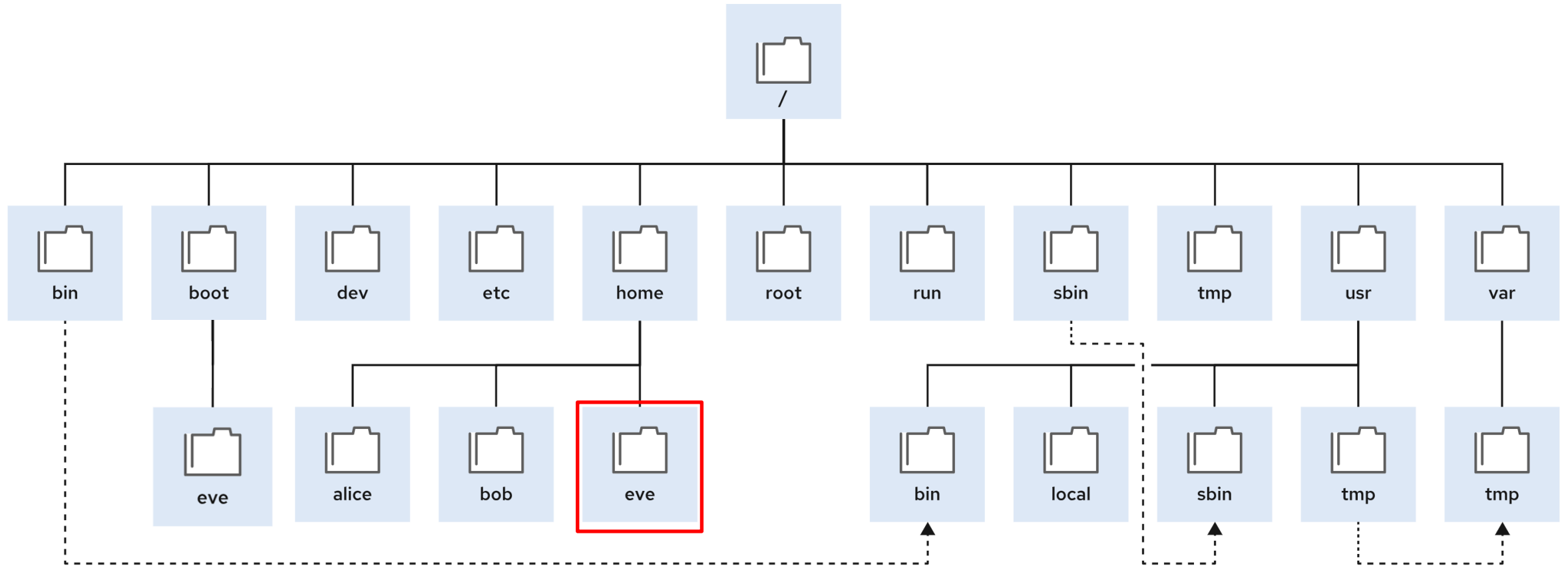


Figure 3.1: Significant file-system directories in Red Hat Enterprise Linux 9

3 Displaying the Current Working Directory

- The *pwd* command (Print Working Directory) display the full path name of the directory where the user is currently located in the file system.
- The user is placed in their home directory (default /home/<user name>) when they initially logs into the system. This is normally the Desktop subfolder of their home directory if they logs into a Graphical User Interface shell.

```
sysadmin@localhost:~$ pwd  
/home/sysadmin
```

- After changing directory to another directory, the present working directory changes

```
sysadmin@localhost:~$ cd Documents/  
sysadmin@localhost:~/Documents$ pwd  
/home/sysadmin/Documents  
sysadmin@localhost:~/Documents$
```

- To return to the home directory, just enter the *cd* command without any arguments

4 Listing Files

- Use the ls command to list the files in the current directory.

```
ls [ OPTION ] ... [ FILE ] ...
```

- ls will also accept an arbitrary number of different path names to attempt to list

```
sysadmin@localhost:~$ ls . Documents /usr/local
.:
Desktop Documents Downloads Music Pictures Public Templates Videos

/usr/local:
bin etc games include lib man sbin share src

Documents:
School      alpha-third.txt  hidden.txt      numbers.txt     spelling.tx
Work        alpha.txt        letters.txt     os.csv          words
adjectives.txt  animals.txt     linux.txt       people.csv
alpha-first.txt food.txt         longfile.txt    profile.txt
alpha-second.txt hello.sh         newhome.txt     red.txt
```

Note: the “.” in the example shown on the left represents the current directory

4 Listing Files (Cont'd)

- Use ***ls -a*** to list all files include hidden files (these are files/directories with names starting with a "."). Hidden files are normally not displayed when using the ***ls*** command.

```
sysadmin@localhost:~$ ls -a
.          .bashrc    .selected_editor  Downloads  Public
..         .cache     Desktop           Music      Templates
.bash_logout .profile   Documents         Pictures   Videos
```

- The ***-d*** option list the directory itself. ***ls*** command will normally list the contents of the directory.

```
sysadmin@localhost:~$ #using the ls command on a directory list the contents of
the directory
sysadmin@localhost:~$ ls /var/log
alternatives.log  bootstrap.log  dist-upgrade  faillog  ndg      wtmp
apt              btmp          dmesg         journal  syslog
auth.log         cron.log      dpkg.log      lastlog  tallylog
sysadmin@localhost:~$ #using the -d option for the ls command list the directory
sysadmin@localhost:~$ ls -d /var/log
/var/log
```


4 Listing Files (Cont'd)

- To learn the details about a file, such as the type of file, permissions, ownership, or the timestamp, perform a *long listing*, use the **-l** option to the **ls** command.

```
sysadmin@localhost:~$ ls -l /var/log/
total 900
-rw-r--r-- 1 root  root  15322 Feb 22 16:32 alternatives.log
drwxr-xr-x 1 root  root   4096 Jul 19  2018 apt
-rw-r----- 1 syslog adm    560 Mar  8 02:17 auth.log
-rw-r--r-- 1 root  root  35330 May 26  2018 bootstrap.log
-rw-rw---- 1 root  utmp      0 May 26  2018 btmp
-rw-r----- 1 syslog adm    293 Mar  8 02:17 cron.log
-rw-r--r-- 1 root  adm   85083 Feb 22 16:32 dmesg
-rw-r--r-- 1 root  root 351960 Jul 19  2018 dpkg.log
-rw-r--r-- 1 root  root  32064 Feb 22 16:32 faillog
```

- The first column of the output list the file type. The file types are:

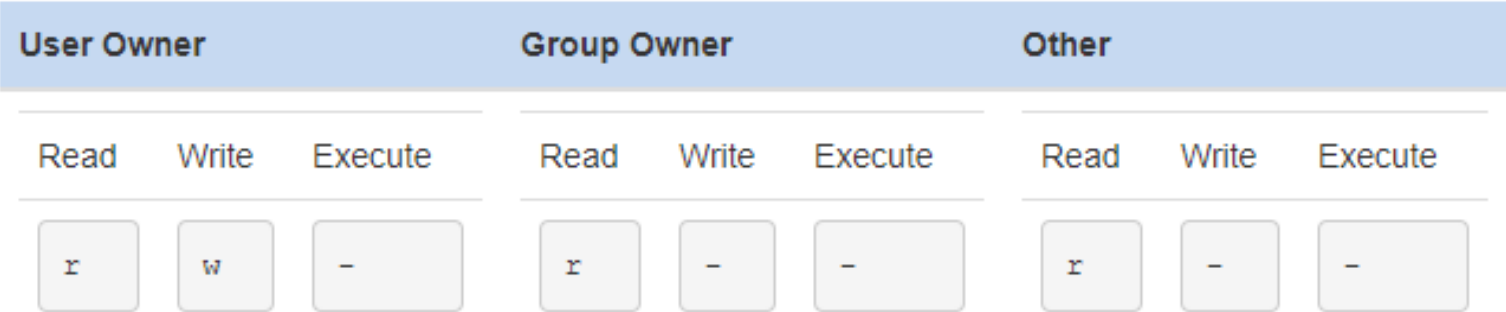
Symbol	File Type	Description
d	directory	A file used to store other files
-	regular file	readable files, images files, binary files, and compressed files.
l	symbolic link	points to another file

4 Listing Files (Cont'd)

- The next nine characters demonstrate the permissions of the file. Permissions indicate how certain users can access a file.

```
sysadmin@localhost:~$ ls -l /var/log/
total 900
-rw-r--r-- 1 root  root  15322 Feb 22 16:32 alternatives.log
drwxr-xr-x 1 root  root   4096 Jul 19  2018 apt
-rw-r----- 1 syslog adm    560 Mar  8 02:17 auth.log
-rw-r--r-- 1 root  root  35330 May 26  2018 bootstrap.log
-rw-rw---- 1 root  utmp     0 May 26  2018 btmp
-rw-r----- 1 syslog adm    293 Mar  8 02:17 cron.log
-rw-r--r-- 1 root  adm  85083 Feb 22 16:32 dmesg
-rw-r--r-- 1 root  root 351960 Jul 19  2018 dpkg.log
-rw-r--r-- 1 root  root  32064 Feb 22 16:32 faillog
```

- The permissions are read r, write w, and execute x. Breaking this down is the diagram below. The topic of permissions is covered in a later lesson.



4 Listing Files (Cont'd)

- The next number is the number of hard links count – the number of directories that links to this file.

```
sysadmin@localhost:~$ ls -l /var/log/
total 900
-rw-r--r-- 1 root root 15322 Feb 22 16:32 alternatives.log
drwxr-xr-x 1 root root 4096 Jul 19 2018 apt
-rw-r----- 1 syslog adm 560 Mar 8 02:17 auth.log
-rw-r--r-- 1 root root 35330 May 26 2018 bootstrap.log
-rw-rw---- 1 root utmp 0 May 26 2018 btmp
-rw-r----- 1 syslog adm 293 Mar 8 02:17 cron.log
-rw-r--r-- 1 root adm 85083 Feb 22 16:32 dmesg
-rw-r--r-- 1 root root 351960 Jul 19 2018 dpkg.log
-rw-r--r-- 1 root root 32064 Feb 22 16:32 faillog
```

- The next two columns list the user and group owners of the file / directory. In the example above, the file cron.log is owned by the user syslog and the usergroup adm.
- The next number to the right is the size of the file in bytes.
- This is followed by the last modified timestamp and finally the full name of the file/directory. If a file/directory is symbolically linked to another file, the linked file is also shown here.

```
lrwxrwxrwx 1 root root 15 Nov 20 05:52 stderr -> /proc/self/fd/2
```

4 Listing Files (Cont'd)

- The -t option sorts the file and directories by last modified dates which is useful for administrators who wish to find out which file/directory was most recently updated.
- The -R option list the contents of all subdirectories and their subdirectories till there are no more subdirectories found hence effectively listing all files and directories within the directory specified in the ls command argument recursively.
- In the example on the right, the home directory of the sysadmin account is listed recursively, in long format and also each directory/subdirectory's contents is listed sorted by last modified date.

```
sysadmin@localhost:~$ ls -lR $HOME
/home/sysadmin:
total 32
drwxr-xr-x 2 sysadmin sysadmin 4096 Apr 24 2019 Desktop
drwxr-xr-x 4 sysadmin sysadmin 4096 Apr 24 2019 Documents
drwxr-xr-x 2 sysadmin sysadmin 4096 Apr 24 2019 Downloads
drwxr-xr-x 2 sysadmin sysadmin 4096 Apr 24 2019 Music
drwxr-xr-x 2 sysadmin sysadmin 4096 Apr 24 2019 Pictures
drwxr-xr-x 2 sysadmin sysadmin 4096 Apr 24 2019 Public
drwxr-xr-x 2 sysadmin sysadmin 4096 Apr 24 2019 Templates
drwxr-xr-x 2 sysadmin sysadmin 4096 Apr 24 2019 Videos

/home/sysadmin/Desktop:
total 0

/home/sysadmin/Documents:
total 1100
drwxr-xr-x 5 sysadmin sysadmin 4096 Apr 24 2019 School
drwxr-xr-x 2 sysadmin sysadmin 4096 Apr 24 2019 Work
-rw-r--r-- 1 sysadmin sysadmin 39 Apr 24 2019 adjectives.txt
```

5 Creating and Modifying Files

- The *touch* command performs two functions. It can create an empty file or update the modification timestamp on an existing file. The first figure below shows an example of the update of modification timestamp where the argument to the *touch* command is an existing file.

```
sysadmin@localhost:~$ ls -l Documents/red.txt
-rw-r--r-- 1 sysadmin sysadmin 51 Feb 22 16:32 Documents/red.txt
sysadmin@localhost:~$ touch Documents/red.txt
sysadmin@localhost:~$ ls -l Documents/red.txt
-rw-r--r-- 1 sysadmin sysadmin 51 Mar  8 02:59 Documents/red.txt
```

- An example when the argument is a file that does not exist.

```
sysadmin@localhost:~$ touch newfile
sysadmin@localhost:~$ ls
Desktop    Downloads  Pictures   Templates  newfile
Documents  Music      Public     Videos
sysadmin@localhost:~$ ls -l newfile
-rw-rw-r-- 1 sysadmin sysadmin 0 Mar  8 03:00 newfile
```

6 Copying Files

- The **cp** command is used to *copy* files. It takes at least two arguments: the first argument is the path to the file to be copied and the second argument is the path to where the copy will be placed. The files to be copied are sometimes referred to as the *source*, and the location where the copies are placed is called the *destination*.
- The example below shows how the **cp** command works. It copies the file `/etc/services` (the source file) to the home directory `~` (the destination directory). Notice that the `services` file is now copied to the home directory and the new file is named `services` as well.

```
sysadmin@localhost:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
sysadmin@localhost:~$ cp /etc/services ~
sysadmin@localhost:~$ ls
Desktop    Downloads  Pictures  Templates  services
Documents  Music      Public    Videos
```

6 Copying Files (Cont'd)

- This next example shows another way that the *cp* command can be used. In this case, the destination is the destination directory with a new file name specified. The result is that a copy of the *services* file is created in the home directory but the file is given a new name *ports*.

```
sysadmin@localhost:~$ cp /etc/services ~/ports
sysadmin@localhost:~$ ls
Desktop      Downloads  Pictures   Templates  ports
Documents   Music      Public     Videos    services
```

- To copy a directory, use the *-R* option to recursively copy all the contents of the directory. *-v* is a verbose option to show more detailed output of the copying process.

```
sysadmin@localhost:~$ cp -R -v /etc/perl ~
'/etc/perl' -> '/home/sysadmin/perl'
'/etc/perl/CPAN' -> '/home/sysadmin/perl/CPAN'
'/etc/perl/Net' -> '/home/sysadmin/perl/Net'
'/etc/perl/Net/libnet.cfg' -> '/home/sysadmin/perl/Net/libnet.cfg'
```

6 Copying Files (Cont'd)

- To run the `cp` command successfully, you require the *execute* permission to access the directory where the source file is located and the *read* permission for the file you are trying to copy. You will also need to have the *write* and *execute* permissions on the directory where you want to put the copied file.

Command	File Type	Permission
<code>cp</code>	Source Directory	Execute
<code>cp</code>	Source File	Read
<code>cp</code>	Destination Directory	Write, Execute

7 Moving Files

- Use the *mv* command to move a file from one path name in the filesystem to another. When you move a file, its like creating a copy of the original file with a new path name and then deleting the original file.
- If you move a file from one directory to another, and you don't specify a new name for the file, then it will retain its original name. For example, the following will move the *~/Documents/red.txt* file into the home directory and the resulting file name will be *red.txt*

```
sysadmin@localhost:~/Documents$ mv red.txt ~
sysadmin@localhost:~/Documents$ ls ~
Desktop      Downloads   Pictures   Templates  red.txt
Documents    Music       Public     Videos
```

- Moving a file within the same directory is to effectively rename the file.

```
sysadmin@localhost:~/Documents$ mv people.csv founders.csv
```

7 Moving Files (Cont'd)

- To move a directory to another location, you do not need to a recursion option. Everything within the directory is automatically moved when the *mv* command is executed.

```
sysadmin@localhost:~/Documents$ cd
sysadmin@localhost:~$ ls Documents/School
Art Engineering Math
sysadmin@localhost:~$ mv Documents/School/Engineering Documents/Work
sysadmin@localhost:~$ ls Documents/Work
Engineering
```

- Moving a file using the *mv* command requires the *write* and *execute* permissions on both the origin and destination directories.

Command	File Type	Permission
<i>mv</i>	Source Directory	Write, Execute
<i>mv</i>	Destination Directory	Write, Execute

8 Deleting Files

- The **rm** command is used to remove files and directories

```
rm [OPTION]... [FILE]...
```

- Without any options, the **rm** command is typically used to remove regular files:

```
sysadmin@localhost:~$ cd Documents
sysadmin@localhost:~/Documents$ rm alpha.txt
sysadmin@localhost:~/Documents$ ls alpha.txt
ls: cannot access alpha.txt: No such file or directory
```

- Deleting a file with the **rm** command requires the *write* and *execute* permissions on its parent directory. Regular users typically only have this type of permission in their home directory and its subdirectories.

Command	File Type	Permission
rm	Parent Directory	Write, Execute

9 Creating Directories

- The *mkdir* command allows you to create (make) a directory. Creating directories is an essential file management skill, since you will want to maintain some functional organization with your files and not have them all placed in a single directory.

```
mkdir [OPTION]... DIRECTORY...
```

- Use the following command to create a *bin* directory in your home directory

```
sysadmin@localhost:~$ mkdir ~/bin
sysadmin@localhost:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos  bin
```

- Directories are often described in terms of their relationship to each other. If one directory contains another directory, then it is referred to as the *parent* directory. The subdirectory is referred to as a child directory. In the following example, /home is the parent directory and the /sysadmin is the child directory

```
sysadmin@localhost:~$ pwd
/home/sysadmin
```

9 Creating Directories (Cont'd)

- When creating a child directory, its parent directory must first exist. If you attempt to create a directory named *john* inside of a non-existent */home/sysadmin/student* directory, the system would generate an error

```
sysadmin@localhost:~$ mkdir /home/sysadmin/student/john
mkdir: cannot create directory '/home/sysadmin/student/john': No such file or directory
```

- Use the `-p` option to overcome this limitation

```
sysadmin@localhost:~$ mkdir -p /home/sysadmin/student/john
sysadmin@localhost:~$ ls -R student
student:
john

student/john:
```

9 Creating Directories (Cont'd)

- To create a directory with the `mkdir` command, you must have the *write* and *execute* permission on the parent of the proposed directory. For example, to create the `/etc/test` directory requires the *write* and *execute* permissions in the `/etc` directory.

Command	File Type	Permission
<code>mkdir</code>	Parent Directory	Write, Execute

10 Removing Directories

- The *rm* command is used to remove empty directories

```
rm -r [OPTION]... DIRECTORY...
```

- Use the *-r* option with the *rm* command to remove directory paths which all contain empty directories.

```
sysadmin@localhost:~$ rm -r /home/sysadmin/student
sysadmin@localhost:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
```

- To delete a directory with the *rm* command, you must have the *write* and *execute* permission on the parent directory.

Command	File Type	Permission
<i>rm</i>	Parent Directory	Write, Execute

11 Getting Help

- You can get online help for a particular command by using the *man* (manual) facility. View the man pages for a command by typing

```
man command
```

```
sysadmin@localhost:~$ man ls
```

```
LS(1)                                User Commands                                LS(1)

NAME
  ls - list directory contents

SYNOPSIS
  ls [OPTION]... [FILE]...

DESCRIPTION
  List information about the FILES (the current directory by default).
  Sort entries alphabetically if none of -cftuvSUX nor --sort is speci-
  fied.

  Mandatory arguments to long options are mandatory for short options
  too.

  -a, --all
      do not ignore entries starting with .

  -A, --almost-all
      do not list implied . and ..

  --author
      print the author of each file

Manual page ls(1) line 1 (press h for help or q to quit)
```

Type “q” to exit the man page
Space or “f” to move forward by a page
“b” to move backward by a page
“j” or down arrow to move down by a line
“k” or up arrow to move up by a line
“h” for help on different short-cut keys

11.1 Sections within Man Pages

Name	Description
NAME	Provides the name of the command and a very brief description <pre>NAME ls - list directory contents</pre>
SYNOPSIS	A brief summary of the command or function's interface. A summary <pre>SYNOPSIS ls [OPTION]... [FILE]...</pre> DESCRIPTION <pre>List information about the FILES (the current directory by default). Sort entries alphabetically if none of -cftuvSUX nor --sort is speci- fied.</pre>
DESCRIPT ION	

11.1 Sections within Man Pages (Cont'd)

Name	Description
OPTIONS	Lists the options for the command as well as a description of how they are used. Often this information is found in the DESCRIPTION section and not in a <pre>-a, --all do not ignore entries starting with . -A, --almost-all do not list implied . and .. --author with -l, print the author of each file -b, --escape print C-style escapes for nongraphic characters --block-size=SIZE scale sizes by SIZE before printing them; e.g., '--block-size=M' prints sizes in units of 1,048,576 bytes; see SIZE format below</pre>

11.1 Sections within Man Pages (Cont'd)

Name	Description
AUTHOR	Provides the name of the person who created the man page and (sometimes) how to contact the person. <pre>AUTHOR Written by Richard M. Stallman and David MacKenzie.</pre>
REPORTING BUGS	<pre>REPORTING BUGS GNU coreutils online help: <http://www.gnu.org/software/coreutils/> Report ls translation bugs to <http://translationproject.org/team/></pre>
COPYRIGHT	Provides basic copyright information
SEE ALSO	Provides you with an idea of where you can find additional information. This often includes other commands that are related to this command.

11.2 Searching within a Man Page

- You can search within a man page for a particular term by typing the / character followed by the term and hitting the enter key. The first occurrence of the search term downwards from the current location in the man page displayed.
- To find the next occurrence, press N
- To find the previous match, press Shift+N.

11.3 Man Page Sections

By default, there are nine sections of man pages:

- Executable programs or shell commands
- System calls (functions provided by the kernel)
- Library calls (functions within program libraries)
- Special files (usually found in `/dev`)
- File formats and conventions, e.g. `/etc/passwd`
- Games
- Miscellaneous (including macro packages and conventions), e.g. `man(7)`, `groff(7)`
- System administration commands (usually only for root)
- Kernel routines [non-standard]

When you use the `man` command, it searches each of these sections in order until it finds the first match. For example, if you execute the command `man cal`, the first section (Executable programs or shell commands) is searched for a man page called `cal`. If not found, then the second section is searched. If no man page is found after searching all sections, you will receive an error message:

11.4 Specifying a Section

You can specifically open the man page of a command in a particular section by using the following command

```
man <section number> command
```

For example, you can open the man page in section 5 (file formats and conventions) of the passwd command to better understand the format of the /etc/passwd file.

```
sysadmin@localhost:~$ man 5 passwd
```

```
sysadmin@localhost:~$
```

PASSWD(5)

File Formats and Conversions

PASSWD(5)

NAME

passwd - the password file

DESCRIPTION

/etc/passwd contains one line for each user account, with seven fields delimited by colons (":"). These fields are:

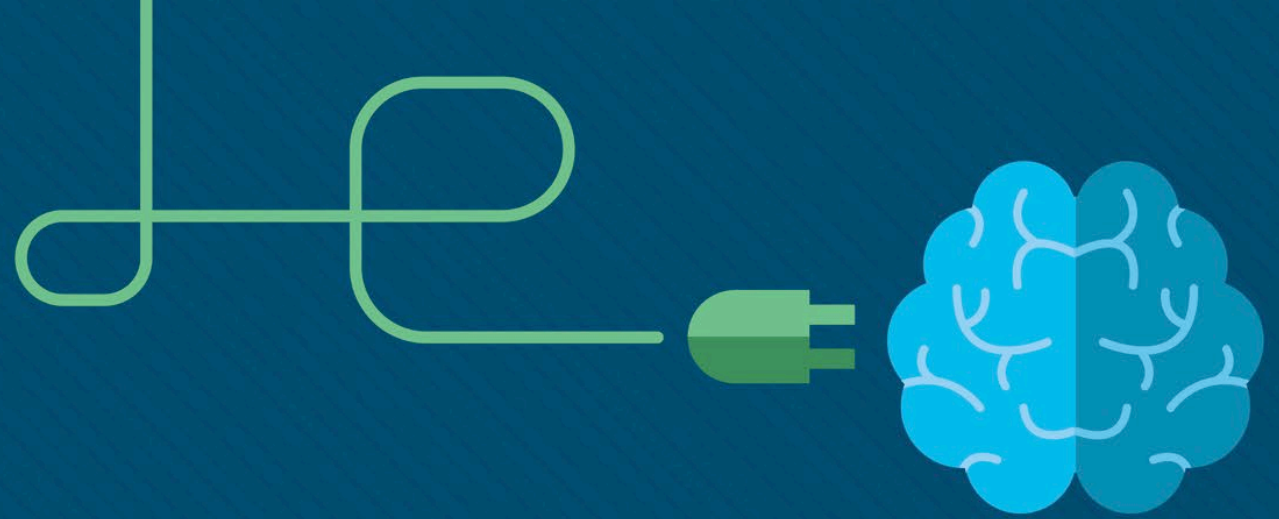
11.5 Search for a man page by keyword

If you not sure what is the command, you can search for relevant man pages by keyword using the -k option for the man command.

```
man -k <keyword>
```

For example, what if you knew you wanted a man page that displays how to change your password, but you didn't remember the exact name? You could run the command `man -k password:`

```
sysadmin@localhost:~$ man -k password
chage (1)           - change user password expiry information
chgpaswd (8)        - update group passwords in batch mode
chpaswd (8)         - update passwords in batch mode
cpgr (8)            - copy with locking the given file to the password or gr..
cppw (8)            - copy with locking the given file to the password or gr..
```



Lab

RH124 Chapter 3.11 Lab: Managing files from the Command Line

RH124 Chapter 4.3 Lab: Get Help in Red Hat Enterprise Linux



Practice Exercises

1 File-System Hierarchy & Navigation

- List the major directories under / and explain their purposes.
- Navigate to /etc and list its contents.
- Use an absolute path to display the contents of /var/log.

2 Absolute & Relative Paths

- Use pwd to check your current working directory.
- Navigate to /home, then use a relative path to move back to your home directory.
- What happens when you run cd .. multiple times?

3 Listing & Understanding Files

- Use ls to list files in /usr/bin. What does ls -l add?
- Show all hidden files in your home directory.
- Identify file types (d, -, l) using ls -l.

4 File Management

- Create a directory named practice_dir in your home directory.
- Inside it, create an empty file called testfile.txt using touch.
- Copy testfile.txt to a new file backup.txt in the same directory.
- Move backup.txt to /tmp.
- Remove testfile.txt from practice_dir.

5 Getting Help

- Use man ls to check its documentation.
- Search for all manual pages related to password.
- Find the passwd command under section 5 of the manual.